Robust PCA for Video Background - Foreground Decomposition

July 3, 2026

Contents

1 Introduction **2**

2 Mathematical Background **3**
2.1 SVD and PCA as a Common Framework . . . 3
2.2 Nuclear Norm (Rank Surrogate) . . . 3
2.3 ℓ_1 norm (Sparsity Surrogate) . . . 3
2.4 RPCA Objective Function . . . 3
2.5 Proximal Operators . . . 4

3 Methodology **4**
3.1 Data Preprocessing . . . 4
3.2 RPCA using IALM . . . 4
3.2.1 IALM . . . 4
3.2.2 Primal Variable Update Step . . . 5
3.2.3 Dual Variable and Penalty Update Steps . . . 5
3.2.4 Algorithm . . . 5
3.2.5 Initialization and Hyperparameters . . . 6

4 Results and Discussion **6**
4.1 Metrics . . . 6
4.1.1 Background Metrics . . . 6
4.1.2 Foreground Metrics . . . 7
4.2 Convergence Analysis . . . 8

5 Conclusion **9**

Abstract

This report investigates video background-foreground separation by formulating the problem as a low-rank matrix decomposition task. Three methods: Truncated Singular Value Decomposition (SVD), Principal Component Analysis (PCA), and Robust Principal Component Analysis (RPCA). These methods are evaluated on the Background Models Challenge (BMC) dataset using a common observation matrix constructed from vectorised video frames. While SVD and PCA recover the background through low-rank approximation, they are unable to explicitly distinguish moving foreground from background variations, often resulting in ghosting artifacts. RPCA addresses this limitation using the Principal Component Pursuit (PCP) formulation, which decomposes the observation matrix into a low-rank background and a sparse foreground, solved via the Inexact Augmented Lagrangian Method (IALM). Performance is evaluated using background reconstruction metrics (MSE, PSNR, SSIM and Frobenius error) and foreground detection metrics (Precision, Recall, F1-score, Specificity and Accuracy). On the synthetic BMC sequence, RPCA converges to a rank-1 background with over 95% foreground sparsity, whereas the real-world sequence requires a rank-25 background with only 18% foreground sparsity, reflecting violations of the low-rank assumption. These results highlight both the effectiveness of RPCA under favourable conditions and the need for dynamic extensions when modelling complex, non-stationary scenes.

1 Introduction

Singular Value Decomposition (SVD) and Principal Component Analysis (PCA) are foundational tools for dimensionality reduction and data compression. Given a data matrix $M \in \mathbb{R}^{m \times n}$, SVD factorises it to $M = U\Sigma V^T$. Using the Eckart - Young theorem, the best k rank approximation of M, denoted by $L = U_k \Sigma_k V_k^T$ can be obtained by truncating the decomposition to its largest k singular value. This is equivalent to least square minimisation i.e., minimize the Frobenius norm (the square root of the sum of the absolute squared elements of a matrix) between data matrix M and low-rank approximation matrix L. Similarly, PCA also applies this, after mean-centering the data, such that the low-rank approximation captures directions of maximum variation with respect to the average sample.

Truncated SVD and PCA attempt to do this by solving the same optimisation problem i.e.,

$$\begin{aligned} \min_{L} \quad & \|M - L\|_F \\ \text{s.t.} \quad & \text{rank}(L) \leq k \end{aligned} \tag{1}$$

This is optimal only with respect to the Frobenius norm, i.e. minimise the overall squared reconstruction error. This has two important consequences:

1. There is no mechanism that separates structured foreground from random noise. The residuals $S = M - L$ absorbs everything that the rank k approximation could not explain, whether that is a genuine moving object or sensor noise. Both SVD and PCA are well known to be highly sensitive to outliers, which can distort the singular vectors. This can dominate the squared loss, where the approximated low-rank component ($\hat{L}$) is far from the true background (L_0).
2. Moving objects 'leak' into the background. In practice, this manifests as 'ghosting' i.e., foreground objects partially appear in the recovered L as semi-transparent artifacts, while S becomes a noisy mixture of real motion and background fluctuation, rather than a clean foreground mask (See Figure 1).

To overcome the above-mentioned shortcomings, the principle of Robust PCA (RPCA) is used where qualitatively different kinds of structure in data are treated separately. Essentially, the low-rank component (L) and the sparse component (S) are recovered through different methods rather than using least-square fit. RPCA assumes that the observation matrix M can be decomposed as a sum of low-rank matrix L_0 and a sparse matrix S_0.

$$M = L_0 + S_0 \tag{2}$$

Initially, the rank of L_0 and entries of S_0 are unknown. But recovering them is possible if the below conditions are satisfied:

1. Incoherence of L_0 i.e., L_0 should not be sparse.
2. S_0 itself should not have a low-rank structure i.e., entries of S_0 should not be confined to a single row or column, otherwise S_0 could be absorbed into the column/row space of L_0.

Under these conditions, authors in [Can+09] show that L_0 and S_0 can be recovered exactly using a convex optimisation approach called Principal Component Pursuit (PCP). Unlike SVD and PCA, which optimise reconstruction error, RPCA aims for higher $||M - L||_F$ in exchange of forcing S towards more sparsity.

To cast a background and foreground separation as RPCA, a video is reshaped into observation matrix M by vectorising and flattening the frames as columns (m is the number of pixels $>> n$ is the number of frames), a static or slow-moving

background can be approximated as a low-rank matrix L as consecutive frames are highly correlated. Any deviation from the background (like a moving pedestrian or vehicle or flickering light) show up as sparse residuals (S) on top of the low-rank matrix. As moving objects occupy only a small fraction of pixels in a particular frame, the foreground can be modelled as a sparse matrix S. Solving Equation 6 gives L and S.

The PCP problem above is convex but non-smooth i.e., nuclear norm and l_1 norm are non-differentiable and therefore cannot be solved by ordinary gradient descent. In practice, RPCA problems are instead solved with first-order methods that exploit the fact that the proximal operators of both norms have simple closed forms. In this work, the Inexact Augmented Lagrangian Method (IALM) solver [LCM10] is used. It works by alternatively updating S or L while keeping the other fixed and then updating the dual variable that enforces the constraint in Equation 2. Essentially, the solver:

1. Updates S (keeping L fixed) reduces to 'soft-threholding' i.e., proximal operator of ℓ_1 norm. This shrinks each entry of the residual towards zero and sets sufficiently small entries exactly to zero, thereby directly promoting sparsity.
2. Updates L (keeping S fixed) reduces to 'Singular Value Thresholding (SVT)' i.e., proximal operator of ℓ_1 norm. The residual is decomposed via SVD, its singular values are soft-thresholded, and the matrix is reconstructed from the surviving components.
3. The dual variable Y accumulates constraint violation $Y \approx M - L - S$ across iterations. The increasing penalty parameter μ progressively forces $L + S$ towards M.

In image and video processing, this idea is applied in a wide range of tasks like image compression, denoising, face recognition under varying illumination, and, most relevant to this report, background modelling in video surveillance [Bou+18].

2 Mathematical Background

2.1 SVD and PCA as a Common Framework

Truncated SVD and PCA are often presented as two separate techniques, but they solve the same optimisation problem and differ in exactly one step i.e., whether the data is mean-centred before the decomposition. Given the observation matrix $M \in \mathbb{R}^{m \times n}$, both methods seek the best rank-k approximation L in the least-squares sense, as stated in Equation 1.

PCA solves the identical problem after one preprocessing step. Writing $\bar{m}$ the per-pixel temporal mean (averaged across frames) and $\tilde{M} = M - \bar{m}\mathbf{1}^T$ for the centred matrix, PCA truncates the SVD of $\tilde{M}$ rather than M:

$$\tilde{M} \approx U_k \Sigma_k V_k^T \quad \text{with} \quad L = U_k \Sigma_k V_k^T + \bar{m}\mathbf{1}^T \quad \text{and} \quad S = M - L \tag{3}$$

Centring re-anchors the rank-k approximation to deviations from the average frame rather than to raw pixel brightness. For video, this matters when the background fluctuates around a stable mean (lighting flicker, sensor noise) rather than staying fixed and PCA's principal components describe that fluctuation directly, whereas raw SVD must spend part of its rank budget reconstructing the (already near-constant) brightness level itself.

2.2 Nuclear Norm (Rank Surrogate)

Minimizing the rank of L is non-convex, therefore optimizing it is difficult. So, instead RPCA uses nuclear norm (sum of singular values) as a convex surrogate to promote low-rank. The nuclear norm is defined as the sum of singular values of L:

$$||L||_* = \sum_i \sigma_i(L) \tag{4}$$

2.3 ℓ_1 norm (Sparsity Surrogate)

Similarly, RPCA requires convex surrogate for sparsity S. This is given by ℓ_1 norm (sum of absolute value of components) of sparsity matrix S:

$$||S||_1 = \sum_{i,j} |S_{i,j}| \tag{5}$$

Minimising $||S||_1$ would make several entries of S to be zero thus encouraging sparsity.

2.4 RPCA Objective Function

Combining two surrogate functions gives PCP as seen in Equation 6. This replaces the least squares (reconstruction) objective function that is used in SVD and PCA. Instead here, two separate penalties act on two separate variables, which are joined by the constraint that they must reconstruct the observation matrix M.

It should be noted that the PCP does not optimise the reconstruction error, but it deliberately accepts higher $||M - L||_F$ in order to reduce S, which is as follows:

$$\begin{aligned} \min_{L,S} \quad & \|L\|_* + \lambda \|S\|_1 \\ \text{s.t.} \quad & L + S = M \end{aligned} \tag{6}$$

where $\|L\|_*$ is the nuclear norm (sum of singular values which acts as a convex surrogate for the rank) and $\|S\|_1$ is the l_1 norm (which is convex surrogate for sparsity). The term $\lambda = 1/\sqrt{\max(\mathrm{m}, \mathrm{n})}$ is the regularization weight. Therefore, it is expected for the RPCA method to give a higher Frobenius error as opposed to SVD or PCA. In addition, due to the non-differentiability of the nuclear norm and the ℓ_1 norm, as they are both non-smooth functions, proximal operators are used.

2.5 Proximal Operators

A proximal operator is a mapping that takes a given point x to a point z that minimises the regularised function f. It finds a new point that decreases the objective function while remaining close to the current estimate. It is defined as:

$$\text{prox}_{\lambda,f}(x) = \arg\min_z \left[f(z) + \frac{1}{2\lambda} ||z - x||_2^2 \right] \tag{7}$$

Soft-Thresholding is the proximal operator of ℓ_1 norm i.e., $f(x) = |x|$ for a scalar x. Soft-Thresholding is given as:

$$\text{soft}(x, \lambda) = \text{sign}(x)\max(|x| - \lambda, 0) \tag{8}$$

where $\lambda \geq 0$. The l_1 penalty shrinks the entries to zero if they are less than λ or they are reduced in value otherwise. This step updates S while keeping L fixed in the IALM algorithm.

Another proximal operator that is used is **Singular Value Thresholding (SVT)** for nuclear norm i.e., $f(x) = ||L||_*$. The soft thresholding is applied to a matrix's singular value, instead of the entries present in the original matrix. That is, for a given matrix X, the SVD is $X = U\Sigma V^T$. For $\lambda \geq 0$, SVT (S_λ) on diagonal entries of singular matrix (Σ) is given as:

$$D_\lambda(X) = US_\lambda(\Sigma)V^T \tag{9}$$

As X is decomposed by SVD, the singular values are either reduced by λ or set to zero which removes the component of the matrix entirely. As the rank of matrix here means number of non-zero values, SVT lowers the rank as nuclear norm penalty intends. This step updates L in the IALM algorithm, treating S as fixed.

3 Methodology

Section 2 establishes what to optimise. This section describes how the above methods are optimised.

3.1 Data Preprocessing

In this work, the Background Models Challenge (BMC) dataset is used [Vac+13]. It provides simulated videos, their ground truth and real-world videos to train and evaluate background separation algorithms. Both simulated and real-world videos are converted into an observation matrix $M \in \mathbb{R}^{m \times n}$ using the same preprocessing pipeline to make the results directly comparable. Each frame is converted to greyscale, resized to 90×120 pixels and normalised to [0, 1]. The flattened frame (a vector of length $90 \times 120 = 10800$) is stored as one column of M, therefore n is the number of frames extracted from a chosen time window. The resultant matrix M has pixel intensity as its rows (spatial dimension) and time as its columns (temporal dimension). As consecutive frames of a static-background scene are highly correlated, they span a low-dimensional column space, while moving objects appear as sparse outliers on top of that low-dimensional structure. All three methods operate on exactly this M, no method-specific preprocessing is applied beyond it.

3.2 RPCA using IALM

3.2.1 IALM

As the PCP objective is constrained, it is reformulated as an unconstrained augmented Lagrangian:

$$\mathcal{L}(L, S, Y) = \|L\|_* + \lambda \|S\|_1 \langle Y, M - L - S \rangle \frac{\mu}{2} |M - L - S|_F^2 \tag{10}$$

where $Y \in \mathbb{R}^{m \times n}$ is a matrix of Lagrange multipliers (one per entry of M) that accumulates the constraint violation $M - L - S$, and $\mu > 0$ is a scalar penalty parameter that progressively tightens the constraint. The quadratic term

$\frac{\mu}{2}|M - L - S|_F^2$ penalises deviation from $L + S = M$ and grows with μ, driving $L + S$ toward M over iterations. A generic Lagrange-multiplier algorithm would minimise $\mathcal{L}$ jointly over L and S at each step, but that joint minimisation is itself a hard problem. IALM avoids it by alternating: update S with L fixed, then update L with S fixed, then update Y.

3.2.2 Primal Variable Update Step

Each alternating update reduces to one of the proximal operators derived in Section 2.5.

Updating S with fixed L such that it minimises $\mathcal{L}$ is equivalent to:

$$S^{k+1} = \arg\min_S \left(\lambda||S||_1 + \frac{\mu}{2}||S - (M - L^{(k)} + Y^{(k)}/\mu)||_F^2\right) \tag{11}$$

which is exactly the definition of the proximal operator of $\frac{\lambda}{\mu}\|\cdot\|_1$. By the result from Section 2.5, the closed-form solution is the entry-wise soft-thresholding operator:

$$S^{(k+1)} = S_{\lambda/\mu}\left(M - L^{(k)} + Y^{(k)}/\mu\right) \tag{12}$$

Every entry of the residual $M - L^{(k)} + \mu^{-1}Y^{(k)}$ is reduced toward zero by the threshold λ/μ. Entries whose magnitude falls below the threshold are set exactly to zero, directly producing the sparsity in S that the ℓ_1 penalty was designed to encourage.

Updating L with S fixed minimises $\mathcal{L}$ over L such that:

$$L^{(k+1)} = \arg\min_L \left(||L||_* + \frac{\mu}{2}\left\|L - \left(M - S^{(k+1)} + Y^{(k)}/\mu\right)\right\|_F^2\right) \tag{13}$$

which is the proximal operator of $\frac{1}{\mu}\|\cdot\|_*$. By the result from Section 2.5, the closed-form solution is obtained using Singular Value Thresholding (SVT):

$$L^{(k+1)} = \mathcal{D}_{1/\mu}\left(M - S^{(k+1)} + Y^{(k)}/\mu\right) \tag{14}$$

The argument matrix is first decomposed using the singular value decomposition (SVD). Its singular values are then soft-thresholded with threshold $1/\mu$, after which the matrix is reconstructed from the remaining singular components. Singular values below the threshold are set to zero, thereby reducing $\text{rank}(L)$, which is exactly the mechanism through which the nuclear norm penalty promotes low-rank solutions.

3.2.3 Dual Variable and Penalty Update Steps

After updating both primal variables, the dual variable and penalty parameter are updated as:

$$Y^{(k+1)} = Y^{(k)} + \mu^{(k)}\left(M - L^{(k+1)} - S^{(k+1)}\right) \quad \text{and} \quad \mu^{(k+1)} = \min\left(\rho\,\mu^{(k)}, \mu_{\max}\right) \tag{15}$$

The update of Y accumulates the constraint violation $M - L^{(k+1)} - S^{(k+1)}$, scaled by the current penalty parameter $\mu^{(k)}$. If the decomposition does not yet satisfy $L + S = M$, the dual variable grows in the direction of the residual, thereby shifting the arguments of the soft-thresholding and singular value thresholding operators in the next iteration to encourage better constraint satisfaction.

The penalty parameter is increased at each iteration by multiplying $\mu^{(k)}$ by the factor $\rho > 1$, subject to the upper bound $\mu_{\max}$. As μ increases, the quadratic penalty term places greater emphasis on satisfying the equality constraint. Together, the updates of Y and μ ensure that $L + S \to M$ as the iterations converge.

3.2.4 Algorithm

Input: $M \in \mathbb{R}^{m\times n}$, λ, μ_0, ρ, $\mu_{\max}$, tol, max_iter.

Initialise:

$$L^{(0)} = \mathbf{0} \quad S^{(0)} = \mathbf{0} \quad Y^{(0)} = \mathbf{0} \quad \mu^{(0)} = \mu_0 \tag{16}$$

For $k = 0, 1, 2, \ldots, \text{max_iter} - 1$:

1. Update the sparse component:

$$S^{(k+1)} \leftarrow \mathcal{S}_{\lambda/\mu^{(k)}}\left(M - L^{(k)} + Y^{(k)}/\mu^{(k)}\right) \tag{17}$$

2. Update the low-rank component:

$$L^{(k+1)} \leftarrow \mathcal{D}_{1/\mu^{(k)}}\left(M - S^{(k+1)} + Y^{(k)}/\mu^{(k)}\right) \tag{18}$$

3. Update the dual variable:

$$Y^{(k+1)} \leftarrow Y^{(k)} + \mu^{(k)}\left(M - L^{(k+1)} - S^{(k+1)}\right) \tag{19}$$

4. Update the penalty parameter:

$$\mu^{(k+1)} \leftarrow \min\left(\rho\,\mu^{(k)}, \mu_{\max}\right) \tag{20}$$

5. Compute the residual:

$$r^{(k)} = \frac{\left\|M - L^{(k+1)} - S^{(k+1)}\right\|_F}{\|M\|_F} \tag{21}$$

   If $r^{(k)} < \text{tol}$, stop (converged).

Output: $L^{(k+1)}, S^{(k+1)}$

3.2.5 Initialization and Hyperparameters

The IALM solver requires five hyperparameters: λ, $\mu^{(0)}$, ρ, $\mu_{\max}$ and the convergence tolerance tol. The regularisation weight λ is set to the theoretically grounded value from [Can+09]. The initial penalty parameter $\mu^{(0)}$ is set using the data-driven rule from [LCM10] i.e., $\mu^{(0)} = \frac{1.25}{||M||_2}$ where $||M||_2$ is the operator norm of M. The penalty is then grown geometrically using Equation 20 with growth factor $\rho = 1.2$ and $\mu_{\max} = 10^4$ is the upper bound on the penalty. The growth factor ρ is chosen conservatively, a large value of ρ constrains $L + S = M$ too quickly, before the low-rank and sparsity matrices have been separated adequately. Solver convergence is declared when the residual (Equation 21) falls below tolerance. The solver is capped at at a maximum of 250 iterations.

4 Results and Discussion

4.1 Metrics

Two complementary groups of metrics are reported, measuring different outputs against different references.

4.1.1 Background Metrics

Background quality metrics (Table 1) evaluate the recovered background L against Frame 0 of the observation matrix M, which contains no moving objects and therefore serves as a clean background reference. Each column $l_t \in \mathbb{R}^{\geqslant}$ of L (the recovered background at frame t) is compared against $f_0 \in \mathbb{R}^m$, the vectorised Frame 0, and the resulting per-frame values are averaged across all n frames. The four background quality metrics are defined as follows:

1. **Mean Squared Error (MSE)** measures the average squared pixel-wise deviation between the recovered background and the reference frame. A lower MSE indicates the recovered background is closer, pixel-by-pixel, to the clean reference.

$$\text{MSE} = \frac{1}{n}\sum_{t=1}^{n}\frac{1}{m}\left\|l_t - f_0\right\|_2^2 = \frac{1}{mn}\sum_{t=1}^{n}\sum_{i=1}^{m}\left(L_{it} - f_{0i}\right)^2 \tag{22}$$

2. **Frobenius Error** measures the overall reconstruction discrepancy between the recovered background matrix L and the reference, treating the full matrix as a single object rather than averaging per-frame:

$$\epsilon_F = \frac{||L - f_0 1^T||_F}{||f_0 1^T||_F}, \quad \text{where } \|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2}. \tag{23}$$

Here $\mathbf{f}_0\mathbf{1}^T \in \mathbb{R}^{m\times n}$ is the reference matrix formed by repeating Frame 0 across all n columns. The normalised form makes the error scale-independent and comparable across datasets with different pixel value ranges. Unlike MSE, which averages per pixel per frame, Frobenius error treats the temporal axis and spatial axis symmetrically.

3. **Peak Signal-to-Noise Ratio (PSNR)** expresses background quality in decibels relative to the maximum possible pixel value. For normalised pixel intensities in [0, 1], the peak signal value is 1. PSNR is a logarithmic rescaling of MSE and carries the same information in a more interpretable form. Higher PSNR indicates better background recovery; values above 40 dB are generally considered high quality.

$$\text{PSNR} = 10\log_{10}\left(\frac{1}{\text{MSE}}\right) \text{ dB} \tag{24}$$

4. **Structural Similarity Index Measure (SSIM)** evaluates perceptual similarity between two images by comparing their local luminance, contrast, and structure simultaneously, rather than measuring raw pixel differences. For two image patches x and y with means μ_x and μ_y, variances σ_x^2 and σ_y^2 and covariance σ_{xy}:

$$\text{SSIM}(x,y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)} \tag{25}$$

where $c_1 = (k_1 \cdot L)^2$ and $c_2 = (k_2 \cdot L)^2$ are small stabilising constants with $k1 = 0.01$, $k2 = 0.03$ and $L = 1$ for normalised images. SSIM ranges from [-1, 1] with 1 indicating identical images. It is computed per frame by reshaping each column of L to 90×120 pixels and comparing against Frame 0 reshaped identically, then averaged across frames.

SSIM is a more meaningful metric than MSE or PSNR because two images can have identical MSE while appearing very different to a human observer i.e., one may have uniform low-level noise while the other has a localised artifact. By capturing structural correlation, SSIM better reflects whether the recovered background is visually coherent.

The limitation shared by all background metrics is that Frame 0 is a single reference frame. Any slow background variation across the sequence (lighting drift, sensor noise, shadow movement) is not captured, so these metrics should be read as a relative comparison between methods rather than as absolute background quality scores.

Metric	**Simulated Data**			**Generalized Data**		
	SVD	**PCA**	**Robust PCA**	**SVD**	**PCA**	**Robust PCA**
MSE	0.0001	0.0001	0.00002	0.0044	0.0046	0.0044
Frobenius error	0.0428	0.0414	0.0481	0.0770	0.0736	0.0803
PSNR (dB)	42.2588	42.0864	45.6543	23.7572	23.6953	23.7108
SSIM	0.9870	0.9860	0.9972	0.8362	0.8319	0.8395

Table 1: Background quality metrics (L evaluated against Frame 0 which has no moving objects).

4.1.2 Foreground Metrics

Foreground detection metrics (Table 2) evaluate the recovered foreground S directly against the binary GT masks provided by the BMC dataset [Vac+13]. Since S is continuous-valued, it is binarised using a single global Otsu threshold computed across all frames jointly, yielding a binary estimate $\hat{S}$. This is then compared pixel-by-pixel against the GT mask $G \in \{0,1\}^{m\times n}$, where $G_{it} = 1$ denotes a foreground pixel. The four detection metrics are defined in terms of per-frame true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN), averaged across frames:

1. **Precision** measures how many pixels flagged as foreground are genuinely foreground. A method that is conservative in its foreground predictions will have high Precision but may miss true foreground pixels, reflected in low Recall.

$$\text{Precision} = \frac{TP}{TP + FP} \tag{26}$$

2. **Recall** (also called detection rate) measures how many true foreground pixels are correctly detected. A method that flags everything as foreground will achieve perfect Recall but at the cost of many false alarms, reflected in low Precision. The tension between Precision and Recall is the central trade-off in foreground detection.

$$\text{Recall} = \frac{TP}{TP + FN} \tag{27}$$

3. **Accuracy** measures the fraction of all pixels, both foreground and background, that are correctly classified.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FN + FP} \tag{28}$$

4. F_1 **Score** is the harmonic mean of Precision and Recall, and penalises both false alarms and missed detections simultaneously. Unlike the arithmetic mean, the harmonic mean is dominated by whichever of the two is lower i.e., a method cannot compensate for poor Recall with excellent Precision or vice versa. A trivial detector that labels every pixel as foreground achieves perfect Recall but near-zero Precision, giving a low F1.

$$\text{F}_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{29}$$

5. **Specificity** measures what fraction of true background pixels are correctly identified as background. It is the complement of the false positive rate. High Specificity means the method does not bleed background pixels into the foreground mask. It is particularly informative in scenes where the foreground occupies a small fraction of the frame, since in those cases even modest false positive rates translate to large numbers of incorrectly labelled background pixels.

$$\text{Specificity} = \frac{TN}{TN + FP} \tag{30}$$

Metric	**Simulated Data**			**Generalized Data**		
	SVD	**PCA**	**Robust PCA**	**SVD**	**PCA**	**Robust PCA**
Precision	0.6256	0.6279	0.6125	0.0105	0.0119	0.0106
Recall	0.0573	0.0598	0.0496	0.0110	0.0117	0.0101
Accuracy	0.9922	0.9922	0.9921	0.9889	0.9891	0.9881
F1-Score	0.1036	0.1078	0.0906	0.0089	0.0102	0.0091
Specificity	1.0000	0.9999	1.0000	0.9943	0.9946	0.9936

Table 2: Foreground detection metrics (binarised S evaluated against GT mask G).

4.2 Convergence Analysis

Figure 3 shows the convergence diagnostics for the IALM solver on the simulated and real-world data respectively. Four quantities are tracked across iterations: the relative residual $r^{(k)}$, the rank of $L^{(k)}$, the sparsity of $S^{(k)}$ (fraction of entries set exactly to zero), and the penalty parameter $\mu^{(k)}$.

1. **Relative residual:** In both cases the residual decays monotonically on a log scale, confirming stable convergence of the IALM iterates. On the simulated data convergence is declared at iteration 46; on the real-world data at iteration 70. The slower convergence on real-world data is consistent with a less stationary background requiring more iterations for the dual variable Y to accumulate sufficient constraint violation and drive $L + S$ towardss M. In both cases the residual crosses the tolerance 10^{-7}, with no stagnation or oscillation, confirming that the hyperparameter choices ($\rho = 1.2$ and $\mu_{\max} = 10^4$) are appropriate.

2. **Penalty parameter:** In both sub-figures $\mu^{(k)}$ grows as a straight line on a log scale, confirming the geometric update $\mu^{(k+1)} = \rho\ ,\mu^{(k)}$ with $\rho = 1.2$. The simulated data traverses the range $[10^{-2}, 10^{1}]$ over 46 iterations and the real-world data traverses the same in 70 iterations. A larger number of iterations is required for real-world data due to lighting variation and scene complexity.

3. **Rank of L:** The rank trajectories reveal the most significant qualitative difference between the two datasets. On the simulated data, $\text{rank}(L^{(k)}) = 1$ from the very first iteration and remains exactly 1 throughout all 46 iterations. This means the SVT operator, which soft-thresholds singular values at $1/\mu^{(k)}$, retains only a single non-zero singular value at every step. This is consistent with the synthetic BMC scene having a genuinely static background (consecutive frames are almost perfectly correlated). The low-rank assumption underlying RPCA is therefore exactly satisfied on this data.

   However, on the real-world data the rank trajectory is different. The rank of L begins at 1, then rises in a staircase pattern from around iteration 10, reaching 25 by iteration 45 and stabilising there until convergence. The staircase shape reflects the SVT operator admitting additional singular components as μ grows and the threshold $1/\mu$ falls, thus, progressively finer structure in the background is captured. A final rank of 25 indicates the real-world background has substantially more temporal variation than the synthetic scene. Lighting changes, mild camera noise, and scene non-stationarity contribute additional singular components that cannot be absorbed into a rank-1 subspace. The

background is still low-rank in an absolute sense (rank 25 against a frame count $n \gg 25$), but the low-rank assumption is considerably weaker than on simulated data.

4. **Sparsity of S:** On the simulated data the fraction of zeros in S begins near 0.95 and rises slightly, stabilising just above the 95% reference line by convergence. This is exactly the expected behaviour. Foreground objects occupy a small fraction of pixels in any given frame, so the ℓ_1 penalty drives the large majority of entries in S to zero via soft-thresholding while retaining the sparse foreground signal.

   While, on the real-world data the sparsity trajectory is significantly different. It begins near 1.0 in early iterations (when the threshold λ/μ is large and almost everything is zeroed out), then falls sharply between iterations 10 and 35, stabilising at approximately 0.18. This means around 82% of entries in S are non-zero at convergence. This is a direct consequence of the rank-25 background. The real-world background variation that cannot be captured by a low-rank L is pushed into S as residual. In other words, S is absorbing not only the genuine moving foreground but also the background complexity that L cannot model. This is a manifestation of model mismatch, the low-rank plus sparse decomposition is not a perfect description of the real-world scene, and the solver distributes the unexplained variation into the sparse component. The foreground detection metrics in Table 2 reflect this directly, as a dense S produces many false positive foreground detections when binarised.

5 Conclusion

This work evaluated three matrix decomposition methods i.e., Truncated SVD, PCA, and Robust PCA via IALM for video background-foreground separation on the BMC dataset, using an identical preprocessing pipeline across all three methods.

SVD and PCA both solve the same least-squares optimisation problem and differ only in whether the data is mean-centred before decomposition. Both are theoretically optimal with respect to Frobenius reconstruction error by the Eckart-Young theorem, but this optimality is precisely their limitation for foreground separation.

RPCA addresses this directly by replacing the Frobenius objective with the PCP formulation, which simultaneously promotes low rank via the nuclear norm and sparsity ℓ_1, subject to the hard constraint $L + S = M$. The IALM solver achieves this through alternating proximal updates, with a dual variable and geometrically growing penalty parameter enforcing the decomposition constraint. Because IALM optimizes the PCP objective rather than reconstruction error, a higher Frobenius reconstruction error than SVD or PCA is an expected and acceptable outcome. The relevant measure of success is whether L converges to a clean, low-rank background and S converges to a sparse, visually coherent foreground mask (See Figure 2), which is assessed via the convergence diagnostics (residual decay, rank of L, sparsity of S).

The limitations of this approach are worth noting. First, the static background assumption is the foundation of the entire model. When it is violated, as in the real-world data, both the rank of L and the density of S increase in ways that directly harm detection quality. Second, RPCA requires a full SVD at each of up to 250 IALM iterations, making it considerably more computationally expensive than truncated SVD or PCA, which each require a single partial SVD. For the $10800 \times n$ matrices used here this is tractable, but scaling to full-resolution video or longer sequences would require randomised or incremental SVD methods.

Despite these limitations, the results confirm the theoretical motivation for RPCA. By treating the low-rank and sparse components as structurally distinct objects with their own penalties, RPCA recovers a cleaner background and a more interpretable foreground than least-squares methods, particularly when the data conforms well to the low-rank plus sparse model. The BMC simulated dataset provides a controlled setting where this advantage is clearly demonstrated; the real-world generalisation experiment shows where the model begins to strain and points naturally towards extensions such as online or dynamic RPCA [Bou+18], which can track a time-varying background subspace rather than assuming a globally static one.

Code Availability

The implementation used in this report is publicly available on GitHub.

References

[Can+09] Emmanuel J. Candes et al. *Robust Principal Component Analysis?* 2009. arXiv: 0912.3599 [cs.IT]. URL: https://arxiv.org/abs/0912.3599.

[LCM10] Zhouchen Lin, Minming Chen, and Yi Ma. "The augmented lagrange multiplier method for exact recovery of corrupted low-rank matrices". In: *arXiv preprint arXiv:1009.5055* (2010).

[Vac+13] Antoine Vacavant et al. “A Benchmark Dataset for Outdoor Foreground/Background Extraction”. In: *Computer Vision - ACCV 2012 Workshops*. Ed. by Jong-Il Park and Junmo Kim. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 291–300. ISBN: 978-3-642-37410-4.

[Bou+18] Thierry Bouwmans et al. “On the Applications of Robust PCA in Image and Video Processing”. In: *Proceedings of the IEEE* 106.8 (2018), pp. 1427–1457. DOI: 10.1109/JPROC.2018.2853589.

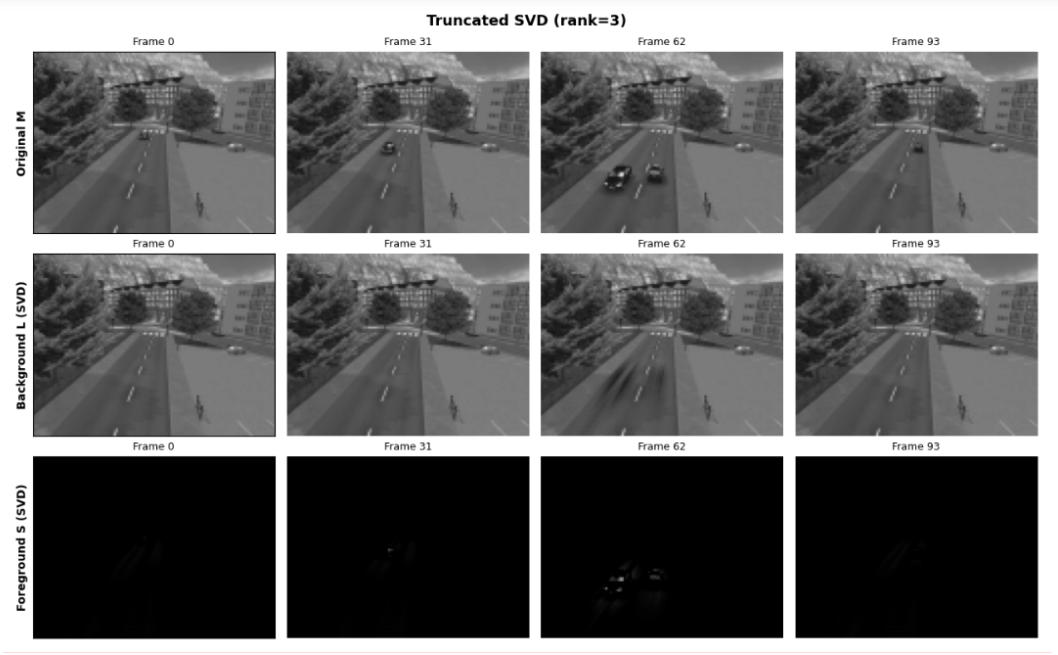
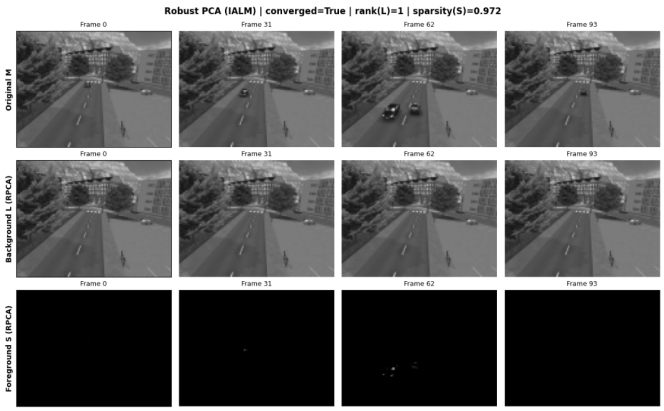
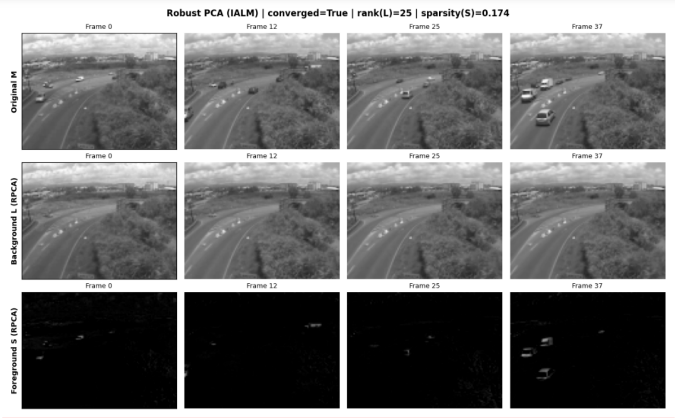


Figure 1: Effect of ghosting when using SVD.

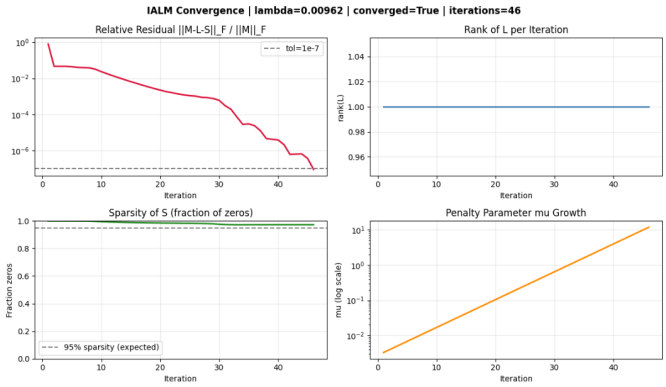


(a) Simulated data.

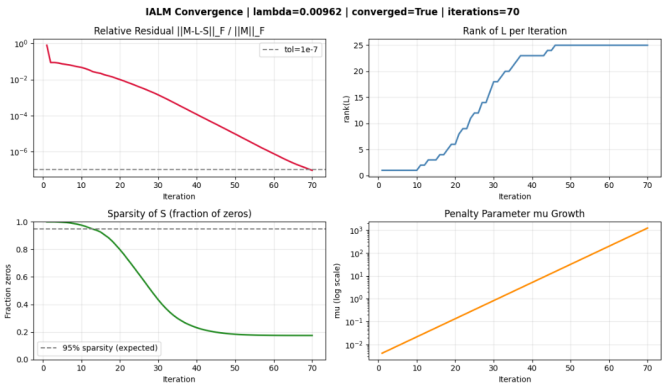


(b) Real data.

Figure 2: Background - foreground separated images using RPCA.



(a) Simulated data.



(b) Real data.

Figure 3: Convergence of IALM optimizer.